

Inventory Management System for B2B SaaS

Overview

"StockFlow" is a B2B inventory management platform aimed at helping small businesses track product quantities across multiple warehouses and manage supplier relationships. This document contains the take-home case study response, including code review, schema design, and API implementation.

Part 1: Code Review & Debugging

Given Code (Flask):

```
@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.json

    product = Product(
        name=data['name'],
        sku=data['sku'],
        price=data['price'],
        warehouse_id=data['warehouse_id']
    )

    db.session.add(product)
    db.session.commit()

    inventory = Inventory(
        product_id=product.id,
        warehouse_id=data['warehouse_id'],
        quantity=data['initial_quantity']
    )

    db.session.add(inventory)
    db.session.commit()

    return {"message": "Product created", "product_id": product.id}
```

1. Issues Identified

- Incorrect Inventory Relationship: Product shouldn't have a direct `warehouse\_id` field. Inventory is a separate many-to-many mapping.
- No SKU Uniqueness Check: Doesn't prevent duplicate SKUs.
- Missing Field Validation: No input validation or error handling.
- No Optional Handling: `initial\_quantity` may be missing, which will raise a KeyError.
- No Transaction Handling: If inventory insert fails, product is still committed.

2. Impacts in Production

- Products may be tied to a single warehouse wrongly.
- Inventory might not be tracked correctly.
- Duplicated SKUs can cause business confusion.
- Partial inserts may cause data inconsistencies.

3. Fixed Version (Node.js, Express):

```
const createProduct = async (req, res) => {
  let connection;

  try {
    const { name, sku, price, warehouse_id, initial_quantity } = req.body;

    connection = await getDbConnection();
    await connection.beginTransaction();

    const [existingSku] = await connection.execute(
      'SELECT id FROM products WHERE sku = ?',
      [sku]
    );

    if (existingSku.length > 0) {
      await connection.rollback();
      return res.status(409).json({ error: 'SKU already exists' });
    }

    const [result] = await connection.execute(
      'INSERT INTO products (name, sku, price, warehouse_id, created_at) VALUES (?, ?, ?, ?, NOW())',
      [name, sku, price, warehouse_id]
    );

    const productId = result.insertId;

    if (initial_quantity !== undefined && initial_quantity >= 0) {
      await connection.execute(
        'INSERT INTO inventory (product_id, warehouse_id, quantity, created_at) VALUES (?, ?, ?, NOW())',
        [productId, warehouse_id, initial_quantity]
      );
    }
    await connection.commit();
  } catch (error) {
    // Error handling
  }
}
```

```

res.status(201).json({
  message: 'Product created successfully',
  product: {
    id: productId,
    name,
    sku,
    price
  }
});

} catch (error) {
  if (connection) await connection.rollback();
  console.error('Error creating product:', error);
  res.status(500).json({ error: 'Failed to create product' });
} finally {
  if (connection) await connection.end();
}
};

```

Part 3: API Implementation

Endpoint: GET /api/companies/:company_id/alerts/low-stock

The logic from `alertController.js` was implemented to compute low-stock alerts with proper SQL joins, calculations for days until stockout, and inclusion of supplier info.

Route Definition (alerts.js)

```
router.get('/companies/:company_id/alerts/low-stock', getLowStockAlerts);
```

Controller Logic (alertController.js)

```

const { QueryTypes } = require('sequelize');
const db = require('../models');

const getLowStockAlerts = async (req, res) => {
  try {
    const { company_id } = req.params;

    // Verify company exists
    const company = await db.Company.findByPk(company_id);
    if (!company) {
      return res.status(404).json({ error: 'Company not found' });
    }

    // query to get low stock alerts

```

```

const alertsQuery = `
SELECT
  p.id as product_id,
  p.name as product_name,
  p.sku,
  w.id as warehouse_id,
  w.name as warehouse_name,
  i.quantity as current_stock,
  p.lowStockThreshold as threshold,
  CASE
    WHEN sales.avg_daily_sales > 0
    THEN FLOOR(i.quantity / sales.avg_daily_sales)
    ELSE NULL
  END as days_until_stockout,
  s.id as supplier_id,
  s.name as supplier_name,
  s.contactEmail as supplier_contact_email
FROM products p
JOIN inventory i ON p.id = i.productId
JOIN warehouses w ON i.warehouseId = w.id
LEFT JOIN product_suppliers ps ON p.id = ps.productId
LEFT JOIN suppliers s ON ps.supplierId = s.id
LEFT JOIN (
  SELECT
    productId,
    warehouseId,
    AVG(quantitySold) as avg_daily_sales
  FROM sales_history
  WHERE created_at >= DATE_SUB(NOW(), INTERVAL 30 DAY)
  GROUP BY productId, warehouseId
  HAVING AVG(quantitySold) > 0
) sales ON p.id = sales.productId AND w.id = sales.warehouseId
WHERE w.companyId = :companyId
  AND i.quantity <= p.lowStockThreshold
  AND p.isActive = 1
  AND (sales.avg_daily_sales > 0 OR sales.avg_daily_sales IS NULL)
ORDER BY (i.quantity / p.lowStockThreshold) ASC, p.name
`;

```

```

// Run the query
const alerts = await db.sequelize.query(alertsQuery, {
  replacements: { companyId: company_id },
  type: QueryTypes.SELECT
});

```

```

// Format response
const formattedAlerts = alerts.map(alert => ({
  product_id: alert.product_id,
  product_name: alert.product_name,
  sku: alert.sku,

```

```

warehouse_id: alert.warehouse_id,
warehouse_name: alert.warehouse_name,
current_stock: alert.current_stock,
threshold: alert.threshold,
days_until_stockout: alert.days_until_stockout,
supplier: alert.supplier_id
? {
  id: alert.supplier_id,
  name: alert.supplier_name,
  contact_email: alert.supplier_contact_email
}
: null
}));

res.json({
  alerts: formattedAlerts,
  total_alerts: formattedAlerts.length
});

} catch (error) {
  console.error('Error fetching low stock alerts:', error);
  res.status(500).json({ error: 'Internal server error' });
}
};

module.exports = { getLowStockAlerts };

```

Edge Cases Handled

- Company not found.
- No sales activity.
- Products with no supplier.
- Division by zero avoided via NULL.

Assumptions Made

- Sales history is tracked daily.
- One supplier per product used in alert (first match).
- Inventory quantity is always positive integer.
- `lowStockThreshold` is always set on product level.

Stock Alert Endpoint – Postman Preview

The image shows the Postman application interface. The top bar includes navigation icons, a search bar, and an 'Infinite' button. The main workspace is titled 'My Workspace' and shows a collection of requests. The selected request is a GET request to 'http://localhost:3000/api/companies/1/alerts/low-stock'. The response is a 200 OK status with a 109 ms response time and 1.2 KB of data. The response body is a JSON object with the following structure:

```
1 {
2   "alerts": [
3     {
4       "product_id": 26,
5       "product_name": "Portable SSD 1TB",
6       "sku": "SKU-SSD1TB",
7       "warehouse_id": 2,
8       "warehouse_name": "Northern Distribution Hub - Delhi",
9       "current_stock": 2,
10      "threshold": 5,
11      "days_until_stockout": 0,
12      "supplier": {
13        "id": 1,
14        "name": "Digital World Distributors",
15        "contact_email": "sales@digitalworld.com"
16      }
17    }
18  ],
19  "total_alerts": 1
20 }
```

The interface also shows a sidebar with 'My first collection' and a 'Create Collection' button. The bottom status bar includes icons for 'Online', 'Find and replace', 'Console', 'Postbot', 'Runner', 'Start Proxy', 'Cookies', 'Vault', 'Trash', and a help icon.